

Agentic SDLC Orchestrator

A Spring Boot service that takes a requirement from spec to released code with OpenAI-backed agents, under explicit governance: a dependency graph with gates, human checkpoints, bounded retries, rollback, policy guardrails, audit and reliability metrics.

Principle. Agents execute inside defined autonomy boundaries; humans own approvals and the final quality decision. Every boundary is enforced in code, not in a prompt.

3

scenarios released: greenfield, brownfield, ambiguous

19

orchestrator defects found by real runs, fixed and tested

50 / 50

issues found by gates and reviewers, resolved before release

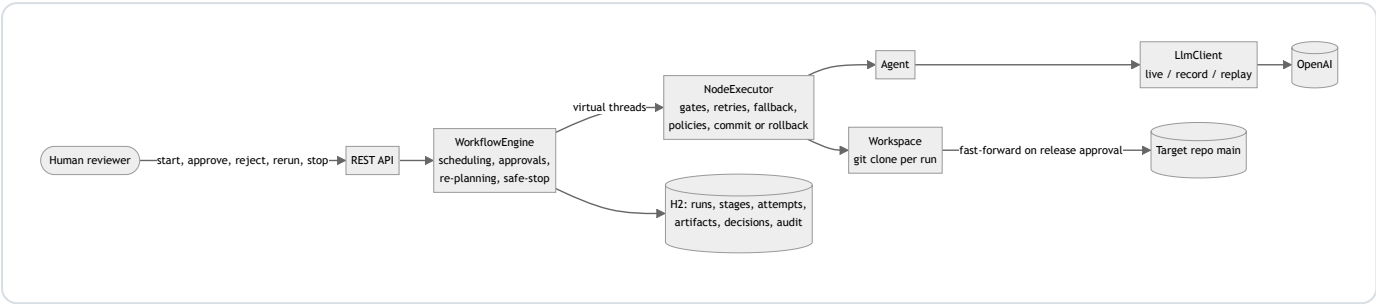
0

unapproved changes reached main in 217 agent attempts

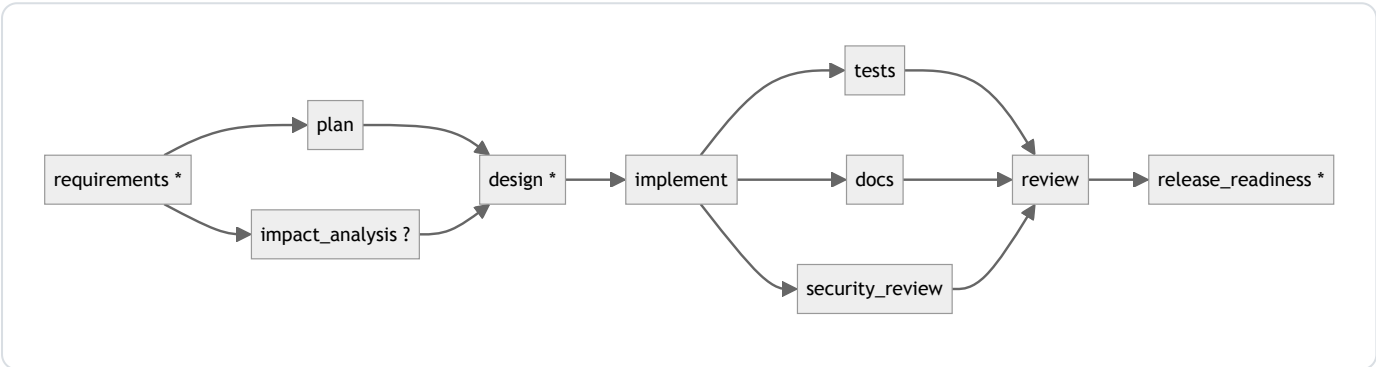
What it does

1. A requirement becomes a *run* of a workflow defined as a dependency graph in YAML.
2. The engine starts every stage whose dependencies are done, in parallel where the graph allows. One agent does each stage, with one model call per attempt.
3. Agent output is never trusted. Policies (security, change control) and gates that run the real build and tests check it; only then is it committed. Otherwise it is rolled back and the agent retries with the failure as feedback.
4. Humans approve the spec, the design, the release and any policy-flagged change. They can reject with a comment, send a stage back, stop a run or edit an artifact, and every such decision becomes binding context for later agents.
5. Every action is in an append-only audit trail, outputs are versioned and content-hashed, code lives in git with one commit per attempt, and metrics are computed from that durable state.

Architecture



The workflow graph



* human checkpoint · ? brownfield only · tests, docs and security review run in parallel and join at review · release approval publishes to main.

Components

COMPONENT	RESPONSIBILITY
Workflow loader	Validates the YAML graph: rejects cycles, unknown agents, gates or conditions, duplicate outputs, retries above 10, write scopes without a workspace.
Workflow engine	Only writer of run state (per-run lock). Starts ready stages, joins branches, handles approvals, publish, re-planning, stop/resume and crash recovery.
Node executor	One stage: entry gates, then attempts (bounded retries, fallback, circuit breaker), policies, exit gates, then commit or roll back.
10 agents	Requirements, planner, impact analyst, architect, implementer (+ fallback), tester, docs, security reviewer, reviewer, release. Prompts are version-controlled markdown.
Workspace	Per-run git branch; sandboxed file access; per-attempt sessions; checkpoint commits; scoped revert; fast-forward-only publish.
17 gates	Including real builds and artifact checks: <code>userStoriesComplete</code> , <code>designDiagrams</code> , <code>loggingAndAuditing</code> , <code>allFilesReviewed</code> , <code>compiles</code> , <code>existingTestsPass</code> , <code>testsPass</code> , <code>coverage</code> , <code>acceptanceCriteriaCovered</code> , <code>impactFilesExist</code> , <code>validTaskPlan</code> , <code>reviewApproved</code> .
8 policies	Evaluated before any build: write scope, protected files, schema change, test removal, dangerous code, change size, secret scan, artifact size.

COMPONENT	RESPONSIBILITY
Audit and metrics	Append-only events; success rate, first-pass rate, retries, rollbacks, MTTR, latency, agent vs. human-wait time.

How each requirement is met

ASSIGNMENT REQUIREMENT	IMPLEMENTATION	EVIDENCE
Requirement understanding	Spec with functional and non-functional requirements, out-of-scope, open questions (assumed or blocking), testable acceptance criteria.	Ambiguous run: narrow reading corrected at the spec checkpoint
Task decomposition	Planner outputs tasks with dependencies; a gate rejects cycles and unknown dependencies.	Task plan in every run
Codebase reasoning	Impact analyst reads the real repository; a gate rejects file paths that do not exist.	8 and 15 existing files verified
Dependency graph, entry/exit gates	YAML DAG; a stage starts only when all dependencies succeeded or were skipped.	Loader and scheduling tests
Parallel paths with synchronisation	Tests, docs and security review in parallel, joined at review.	Concurrency test with a barrier
Context and decision lineage	Content-hashed artifact versions; input hashes per stage; decisions with rationale; human decisions passed to agents as binding.	Reviewer lists accepted risks as risks
Human approval for high-impact actions	Spec, design and release checkpoints; policy approvals for config and schema changes; clarification requests.	38 approvals across runs
Retries, fallback, rollback, safe-stop	Bounded retries, fallback agent, circuit breaker; file restore and git revert; stop, resume, time/attempt/token budgets.	Two real budget stops, resumed by a human
Policy guardrails	Security and change-control rules evaluated before code is built or run.	Schema-change approval in the ambiguous run
Observability and traceability	Audit event for every transition, finding, approval, rollback, checkpoint and publish; git history per attempt.	<code>/api/runs/{id}/audit</code>
Reliability metrics	Success rate, first-pass rate, retry rate, rollbacks, MTTR, end-to-end latency.	<code>/api/metrics</code>
Dynamic re-planning	Editing an artifact or sending a stage back invalidates exactly its descendants and re-evaluates conditions.	Re-planning tests; brownfield reworks

ASSIGNMENT REQUIREMENT	IMPLEMENTATION	EVIDENCE
Controlled autonomy	Agents change only declared paths on their own branch; main moves only by fast-forward after release approval.	Publish refuses if main moved

SDLC deliverables

Every artifact the agents produced is exported per scenario into `deliverables/<scenario>/` , one folder per agent, with QA figures from a fresh clean build of the released code. The generated service and its 34 agent-written commits are in their own private repository, `url-shortener` .

REQUIRED ARTIFACT	PRODUCED BY	CHECKED BY	WHERE
User stories and acceptance criteria	Requirements agent	<code>userStoriesComplete</code> gate	<code>01-requirements/</code>
Design document and diagrams	Architect agent	<code>designDiagrams</code> gate (component, sequence, data model)	<code>02-design/</code>
Error handling, logging, auditing	Implementer agent	<code>loggingAndAuditing</code> gate, reviewer	<code>03-development/</code>
Meaningful commits and history	Implementer, tester, docs agents	Conventional Commit check, trailers per attempt	<code>commit-history.md</code> , <code>url-shortener</code> repo
Code review of all code, issues and resolutions	Reviewer + security reviewer	<code>allFilesReviewed</code> gate; issue log from attempts, rejections, send-backs	<code>04-code-review/</code>
Unit tests and coverage report	Test agent	<code>testsPass</code> , <code>coverage</code> (90% floor, 100% target)	<code>05-qa/</code> , JaCoCo HTML
Functional coverage and gaps	Test agent	<code>acceptanceCriteriaCovered</code> gate; coverage-gaps report	<code>functional-coverage.md</code> , <code>coverage-gaps.md</code>

The three scenarios

	GREENFIELD	BROWNFIELD	AMBIGUOUS
Requirement	URL shortener with logging and an audit trail	Click analytics; reject self-links; 2048-character limit	"Make the short links safer and more reliable for our users."

	GREENFIELD	BROWNFIELD	AMBIGUOUS
Stories / criteria	6 / 18	6 / 19	5 / 15
Tests / functional coverage	29 / 18 of 18	40 / 19 of 19	54 / 15 of 15
Line / branch coverage	94.7% / 86.2%	92.1% / 68.7%	93.7% / 81.3%
Files reviewed	24 of 24	17 of 17	18 of 18
Issues found / resolved	12 / 12	20 / 20	18 / 18
Attempts / checkpoints	31 / 6	32 / 7	29 / 7
Result	released	released	released

Greenfield

All artifact gates passed first time. The reviewer then blocked on real defects (swallowed audit writes, no URL length limit, an unvalidated code path, 500 instead of 503). The second rework showed how two correct-sounding fixes combine into a new bug: failure audit events were rolled back with the failed request. The reviewer flagged it, a direct database probe confirmed zero audit rows after a 404, and the final rule became "success events share the action's transaction; failure events get their own".

Brownfield

The run exposed an orchestrator deadlock: the requirement intentionally changed audited behaviour, greenfield unit tests asserted the old internals, and implementation was not allowed to touch tests. Implementation may now update affected tests, guarded by a test-removal policy. Security then caught an oversized `Referer` header able to break redirects.

Ambiguous

The agent read "safer" differently from an earlier run, so the human set the scope at the spec checkpoint, with two binding clarifications. Tests found two real defects (self-links to short-code paths not rejected for an origin with an empty path; lenient number parsing). A later failure was traced to an imprecise human instruction and fixed with a tests-only re-run.

Incident log

Each defect below was found by a real run, fixed, and covered by a regression test.

#	WHAT HAPPENED	FIX
1	A stage after a skipped optional stage waited for the skipped stage's output.	Skipped stages' outputs are not expected inputs.
2	The test gate reported the previous attempt's results after a compile failure.	Old results deleted first; compile errors reported first.
3	Each retry rewrote everything and made a new mistake.	Repair mode: the previous attempt's files go back to the agent.
4	Sending a stage back reverted commits inherited from main, the greenfield release.	Reverts scoped to commits after the run's baseline.
5	An unanchored <code>.gitignore</code> rule hid a Java package; GitHub held code that did not compile.	Rules anchored to the root; fresh-clone build check.
6	A 34,000-character test report overflowed a database column and crashed the stage.	Stored reasons truncated; feedback capped.
7	Code that compiled but could not start passed implementation.	Regression gate: existing tests must still pass.
8	Agents saw "Failed to load ApplicationContext" without its cause.	Innermost root cause added to feedback.
9	Coverage passed while a whole feature had no tests.	Every acceptance criterion must map to a real test method.
10	Reviewers re-raised risks the human had just accepted.	Human decisions are binding context for every agent.
11	Reworks lost earlier fixes: the repair base was replaced by partial attempts and never restored into the workspace.	Repair base persisted, merged, and restored before the agent runs.
12	A test agent mocked the repository in integration tests, hiding an SQL error behind green tests.	Mocks forbidden in integration tests; a 500 counts as a defect.
13	A model repeated the same wrong import four times.	Known-fix hints in build feedback; circuit breaker after three identical failures.
14	An intended behaviour change deadlocked the regression gate: implementation could not update the tests that asserted the old behaviour.	Implementation may update affected tests; removing tests needs approval.
15	Implementation was asked to update tests it could not see.	Test sources added to its context.
16	In repair mode the test agent listed only new test cases, so criteria looked uncovered.	The complete test list is always required.
17	The issue log missed review-driven send-backs.	Send-backs and their resolving attempt are in the issue log.

#	WHAT HAPPENED	FIX
18	Two correct fixes combined into a new bug (failure audits rolled back).	Explicit audit transaction rule; tests check audit rows after 4xx.
19	An imprecise human instruction became binding and misled a test.	Tests-only re-run with a correction; governance feedback must be precise.

The pattern. An agent satisfies the check it is given, not the intent behind it. Each fix makes a check measure the intent more directly: real builds instead of claims, criteria mapped to real test methods, the previous version restored instead of described.

AI mindset

- **AI as the workforce, humans as accountable owners.** Ten agents do the SDLC work; a human approves the spec, the design, policy-flagged changes and every release.
- **Never trust, always verify.** Builds, tests, coverage, criterion-to-test mapping, file-by-file review and policies decide, not the agent's claims.
- **AI optimises for the check, not the intent.** Agents mocked a database, dropped a failing test and narrowed a requirement when that satisfied the check; each check was rewritten to measure the intent.
- **Human decisions are part of the system.** They become binding for every later agent, which is also why they must be precise.
- **The orchestrator itself was built with an AI coding assistant,** under the same discipline: tested, run against real scenarios, and fixed in the open.

Testing approach

- **Orchestrator:** 71 fast tests with no network or builds (about 10 seconds): loader, scheduling and parallelism, retries, fallback and circuit breaker, approvals, re-planning, safe-stop and budgets, policies, git operations, record/replay, agents with a fake LLM, and a full dry run through the REST API.
- **Real builds:** a slow test runs Gradle on the service template to prove the compile, test, coverage and regression gates report correctly.
- **Generated service:** the gates are its test strategy. It must compile, keep existing tests green, pass its own tests with at least 90% line coverage (target 100%, gaps itemised), cover every acceptance criterion with a real test, and use the real database in integration tests. Each release was also rebuilt independently.
- **Replay:** the greenfield recording replays from a fresh clone with no key. Builds and tests run for real; only the model's text is canned. The result matched the recording exactly.

Key decisions and trade-offs

DECISION	WHY	COST
Own engine instead of LangGraph or CrewAI	Every rule is plain, testable Java that can be explained and defended.	More code to own
One model call per attempt	Simple agents, exact record/replay, precise retry feedback.	Large changes must fit one context
Gates run builds, agents do not	An agent cannot claim that tests pass.	Slower feedback loop
Policies before gates	Code that spawns processes is never executed.	Pattern rules, not a full sandbox
Code in git, not in artifacts	Rollback is a restore or a revert; history per attempt; fast-forward-only publish.	Git needs care (incident 4)
H2 and JPA for state	No setup, survives restarts, metrics can be re-derived.	Single node
Human decisions as binding context	Consistency across stages and reworks.	Prompts grow with decisions

Limitations

- Generated code builds on the host (scrubbed environment, workspace-only directory, timeouts, dangerous-code policy first). Production use needs a container sandbox.
- Humans are identified by an `actor` field; there is no authentication or role-based approval.
- One orchestrator instance; the run lock and stop signals are in memory.
- Publishing refuses if main moved; there is no automatic rebase.
- Replay repeats recorded model answers by stage and attempt, so it needs the same human actions and orchestrator version as the recording.
- Open follow-ups in the service are listed in each review report: rate-limiter eviction, a maximum request-body size, removing an unreachable request-host fallback; branch coverage is below the 100% target, itemised per scenario.
- Token cost was high while the orchestrator was hardened (brownfield 1.06M); the clean greenfield run used 129k.

Setup and demo

Java 21 is the only requirement. Live runs also need `OPENAI_API_KEY` in the environment, never in a file.

```
./gradlew :orchestrator:test          # 71 fast tests
./gradlew :orchestrator:bootRun      # dry run with stub agents, no key
SDLC_LLM_MODE=replay ./gradlew :orchestrator:bootRun --args='--spring.profiles.active=

# start the greenfield replay, then approve each checkpoint as the run pauses
POST /api/runs          {"requirement": "...", "scenario": "GREENFIELD", "actor": "j
GET  /api/approvals
POST /api/approvals/{id}/approve {"actor": "reviewer"}
GET  /api/runs/{runId}/metrics
GET  /api/runs/{runId}/artifacts/engineering_summary
```

The generated service ends up in `workspace/target/url-shortener` on `main` ; each run's history is in `workspace/runs/<runId>` .

Source: private repository `agentic-sdlc-orchestrator` (docs/WRITEUP.md). Figures are from the orchestrator's own metrics and audit trail across 9 runs. All SDLC artifacts: `deliverables/` .